

→ Generics

→ Custom Arraylist

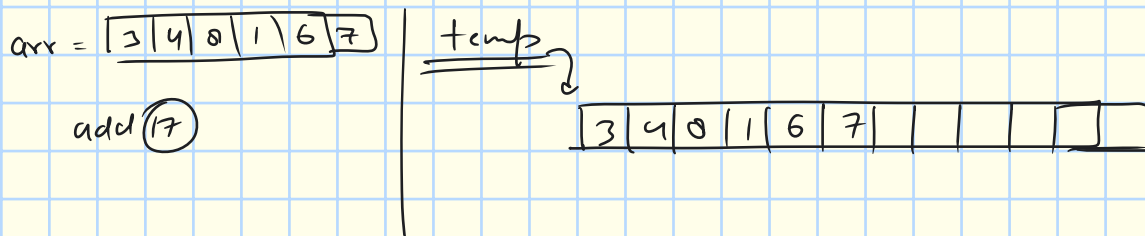
→ Lambda Expression

→ Exceptional handling

→ Object Cloning



Custom ArrayList



Problem I faced that I'm able to create only int type of Custom ArrayList

⇒ I cannot use this code for Strings, float etc.,

⇒ Problem with Custom ArrayList

→ It's only storing int types, but Internal ArrayList can be of type Integer, float, char, String, etc., (Internal ArrayList provide this functionality).

⇒ This functionality is known as Generics (Parametrized)

Generics

in Java refer to parametrized types that allow writing code which work with multiple data types using a single class, interface, or method.

⇒ They improve reusability and ensure type safety at compile time.

- Use type parameters (like <T>) to create flexible & reusable code.
- Prevent runtime errors by enforcing type safety at compile time.

⇒ Why use Generics?

- Before Generics, Java collection like ArrayList or HashMap could store any type of objects, everything was treated as an Object. It had some problems.
- If you added a String to a list, Java didn't remember its type. You had to manually cast it when retrieving. If the type was wrong, it caused a runtime error.
- With Generics, you can specify the type, the collection will hold like ArrayList<String>. Now, Java knows what to expect & it checks at compile time, not at runtime.

⇒ Types of Java Generics

- 1) Generic class is a class that can operate on objects of different types using a type parameter.
- 2) Generic method : we can also write generic methods that can be called with different types of arguments based on the type of arguments passed to the generic method.
 - The compiler handles each method.

```
ArrayList list = new ArrayList();
```

```
list.add(2134);
```

```
list.add("anshul");
```

```
list.add('A');
```

```
System.out.println(list);
```

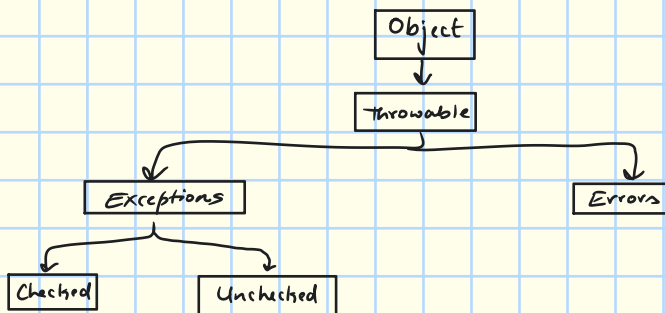
If type safety is missing, can add any type value in list.

Hence, Generic is required for these types of structures.

Exceptional handling

Exceptional handling in Java is a robust mechanism that handles runtime errors, preventing the application from crashing and maintaining a smooth program execution flow.

⇒ Throwable class handles all the exception & error.



⇒ Arithmetic exception.

ex:-

```
int a = 5;  
int b = 0;  
int c = a/b; → // Exception is thrown .... java.lang. ArithmeticException : / by zero.
```

Use try-catch blocks,

```
int a = 5;
```

```
int b = 0;
```

```
try {
```

```
    int c = a/b;
```

```
catch (ArithmeticException e) {
```

```
    System.out.println(e.getMessage());
```

```
} finally {
```

```
    System.out.println("This will always execute");
```

```
}
```

* try → The try block contains code that might throw an exception.

* catch → The catch block handles the exception if it occurs.

* finally → The finally block executes after the try & catch blocks in most situations, whether an exception occurred or not.

• basically used for closing resources such as database connections, open files, or network connection.

⇒ Multiple Catch blocks.

Can use multiple catch lines.

→ Use to catch multiple error

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 0;  
        try {  
            divide(a, b);  
        } catch (ArithmeticException e) {  
            System.out.println(e.getMessage());  
        } catch (Exception e) {  
            System.out.println("normal exception");  
        } finally {  
            System.out.println("this will always execute");  
        }  
    }  
}
```

```

catch (Exception e) {
    System.out.println("normal exception");
} catch (ArithmeticException e) {
    System.out.pr
} finally {
    System.out.pr
}

```

Exception 'java.lang.ArithmeticException' has already been caught
Delete catch for 'java.lang.ArithmeticException' More actions...

java.lang
public class ArithmeticException
extends RuntimeException

Thrown when an exceptional arithmetic condition has occurred. For example, an integer "divide by zero" throws an instance of this class. ArithmeticException objects may be constructed by the virtual machine as if suppression were disabled and/or the stack trace was not writable.
Since: 1.0
openjdk-16 >

```

tic int divide(int

```

Exception $\Rightarrow$ include everything
So, if you write Exception in 1st catch block, then you cannot put any specific exception in other catch blocks

$\Rightarrow$ Can throw exception

```

try {
    divide(a, b);
    // mimicing
    throw new Exception("just for fun");
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
} catch (Exception e) {
    System.out.println("normal exception");
} finally {
    System.out.println("this will always execute");
}

```

$\Rightarrow$ Creating Custom Exception

Code

MyException.java

```

public class MyException extends Exception {
    public MyException (String message) {
        super(message);
    }
}

```

main.java

```

try {
    String name = "Anshul";
    if (name.equals("Anshul")) {
        throw new MyException ("name is Anshul");
    }
} catch (MyException e) {
    pSout(e.getMessage());
}

```

Object cloning

ex:- Human kunal = new Human(34, "Kunal Kush");

Human twin = new Human(kunal); $\rightarrow$ we created copy of kunal

$\Rightarrow$ Creating exact copy of an object.

```

public class main {
    public static void main(String[] args) throws CloneNotSupportedException{
        Human kunal = new Human( name: "Kunal Kushwaha", age: 34);
        Human twin = new Human(kunal);

        Human twin2 = (Human)kunal.clone();

        System.out.println(twin2.name);
    }
}

```

$\hookrightarrow$ but this take more processing time,
Hence use Object Cloning

```

public class Human implements Cloneable { 1 usage
    int age; 3 usages
    String name; 3 usages

    public Human(String name, int age) { no usages
        this.name = name;
        this.age = age;
    }

    public Human(Human other) { no usages
        this.name = other.name;
        this.age = other.age;
    }

    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

```

this cloning is shallow cloning

Shallow copy.

Kunal = { age = 34
name = "Kunal Kushwaha"
arr = [3, 4, 5, 6, 9, 1]

twin = { age = 34
name
arr

Imp:-

In cloning primitive datatypes will be created again, but non-primitive will not going to create a new one it's reference gonna point to original.

So, change made in twin will reflected in original
Hence, it is shallow copy as it made changes in original via changing in copy.

Deep Copy

Kunal = { age = 34
name = "Kunal Kushwaha"
arr = [3, 4, 5, 6, 9, 1]

twin = { age = 34
name = "Kunal Kushwaha"
arr = [3, 4, 5, 6, 9, 1]

